

USER DEFINED FUNCTIONS

What is a function?

1. It is a small program used to do a particular task.
2. It is a small program with in another program.
3. It is a sub program or sub routine.
4. It is a procedure.
5. It is a self-contained block.
6. It is a reusable code component.
7. It is a module.

Advantages:

1. **Modularity**: Dividing program instructions into small pieces.
2. **Simplicity**: It is easy to understand the program instructions.
3. **Reusability**: Just write once, use several times.
4. **Efficiency**: performance is improved.
5. It is easy to identify the errors.

Entire 'C' program is collection of functions.
Hence 'C' is a function oriented structured programming language.

We are having 2 types of functions.

1. Predefined / library / built-in functions

These are the functions provided with software and ready to use.

Eg: printf(), scanf(),...

2. **User defined functions:** They are created by the user.

Eg: sum(), prime(), factorial(),.....

Every user defined function is divided into 3 parts.

1. Function declaration/ proto typing
2. Function calling
3. Function definition

Function declaration/proto typing:

Generally function declaration is conducted before or within the main(), before function calling.

It tells the compiler that we are going to use this kind of function in our program in future.

Syntax:

```
[return_datatype] function_name ( [arguments / parameters] );
```

Eg:

```
void sum();
```

Function calling:

It should be conducted within the main() only.

When a function is invoked (called), the compiler will search for the matching function definition and if it is available then the program execution is jumped from function calling area to the function definition area.

Linking a function call to the function definition is called **binding**. C language supports only static or compile time binding. But C++ supports static and dynamic binding[run time binding].

Without function call, a function never participates in function execution. Hence it is mandatory to call a function in a program.

Syntax:

```
function_name([arguments]);
```

Eg: sum();

Function definition:

It contains the function header and body. The function header should be matched with function declaration.

It is conducted outside the main(). If the definition is conducted before the main(), there is no need of function declaration.

Function header consists of function name and the parameters.

Function body consists of statements related to the task of function.

When a function is invoked, the program execution is shifted from function calling area (main()) to function definition area. After executing the function body, the execution is jumped to the next statement after the function call in main().

Function declaration and function definition should be identical.

Syntax:

Function Header
↗

```
[return data type] Function_name(  
[parameters/arguments] )  
  
{  
    Statements;  
}
```

Function Body

```
[return value;]  
}
```

Here return data type indicates the type of value that the function is returning to main() / called function. It is depended on the return value.

If the sub program is having return statement then it is called **function and without return value it is called **procedure**.**

The **default return value of a function is integer**. Use the keyword **void** [nothing] when the function is not having any return value. Otherwise integer will become the return value.

A function may have any number of return statements. But only one return statement is executed at a time. The statements after the return statement are not executed and gives compile time warning.

Return data type should be matched with return value data type.

Function name is used to identify a function and it should not be predefined.

Arguments / parameters used to carry the values from function calling area to function definition area. They are optional and of two types.

1. **Actual** arguments/parameters.
2. **Formal** arguments/parameters.

The arguments we are sending in function calling at `main()` are called **actual parameters** and the parameters we are using in function definition to receive the actual argument values is called **formal parameters**.

Actual and formal parameter names may be same or different.

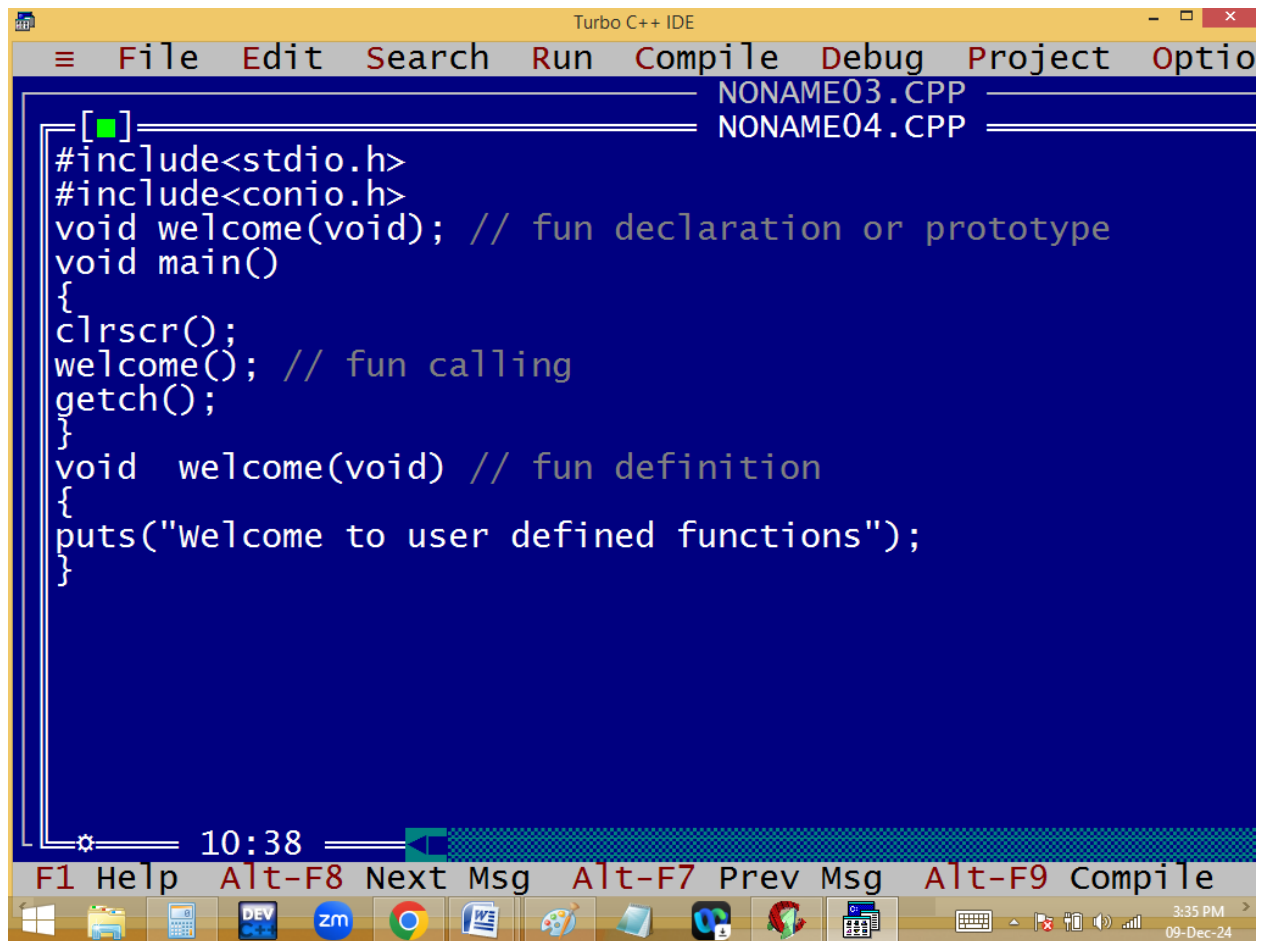
Difference between arguments and parameters is “arguments are nothing but different vehicles which are parked in same parking area and the parking area is the parameters. i.e. arguments are always changed but parameters are fixed.

The calling function arguments should be matched with definition parameters list in quantity, data type and order.

Based on arguments and return values, functions are divided into 4 types.

1. Function without arguments, without return values.
2. Function with arguments and with return values.
3. Function without arguments and with return values.
4. Function with arguments and without return values.

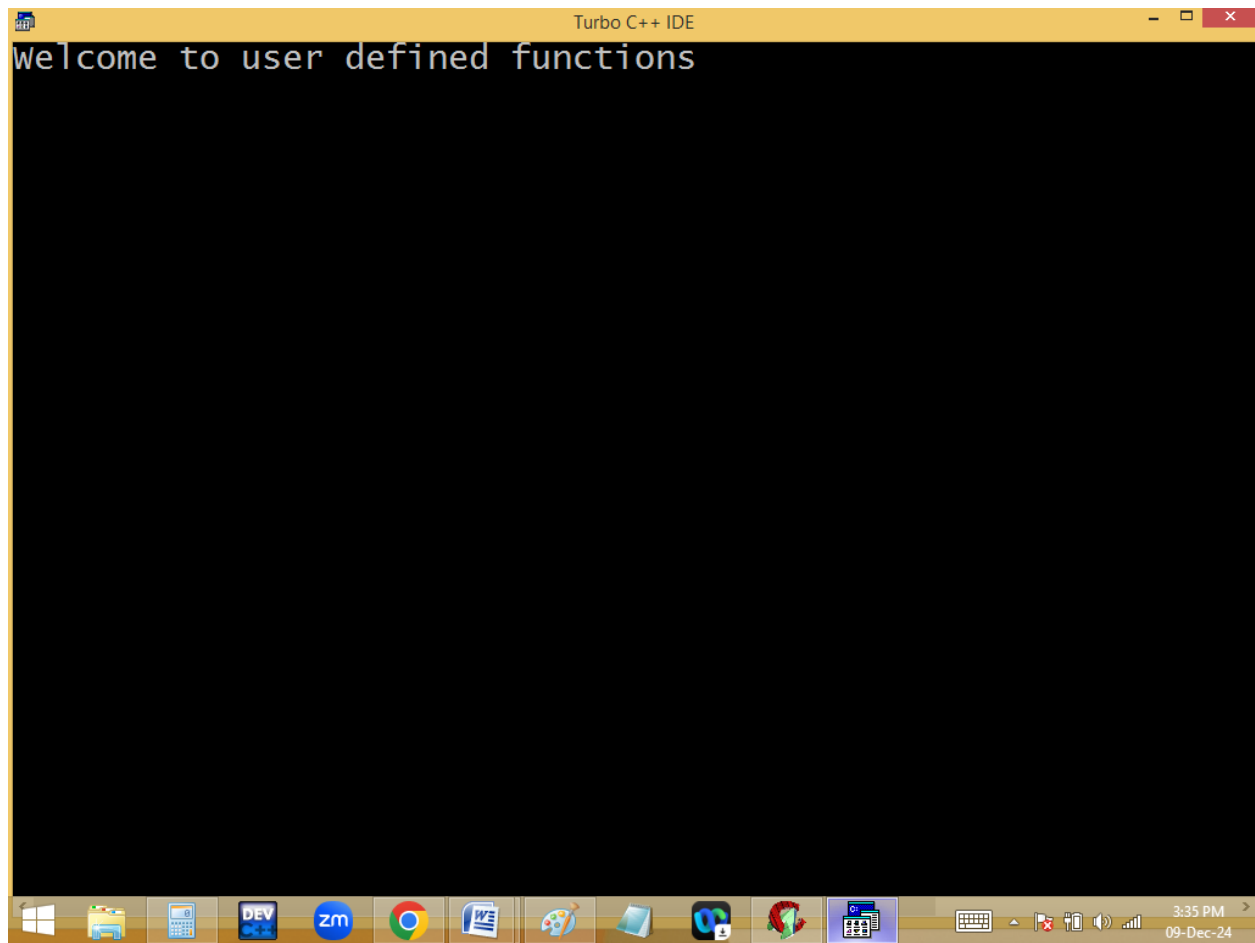
Function without arguments, without return value:

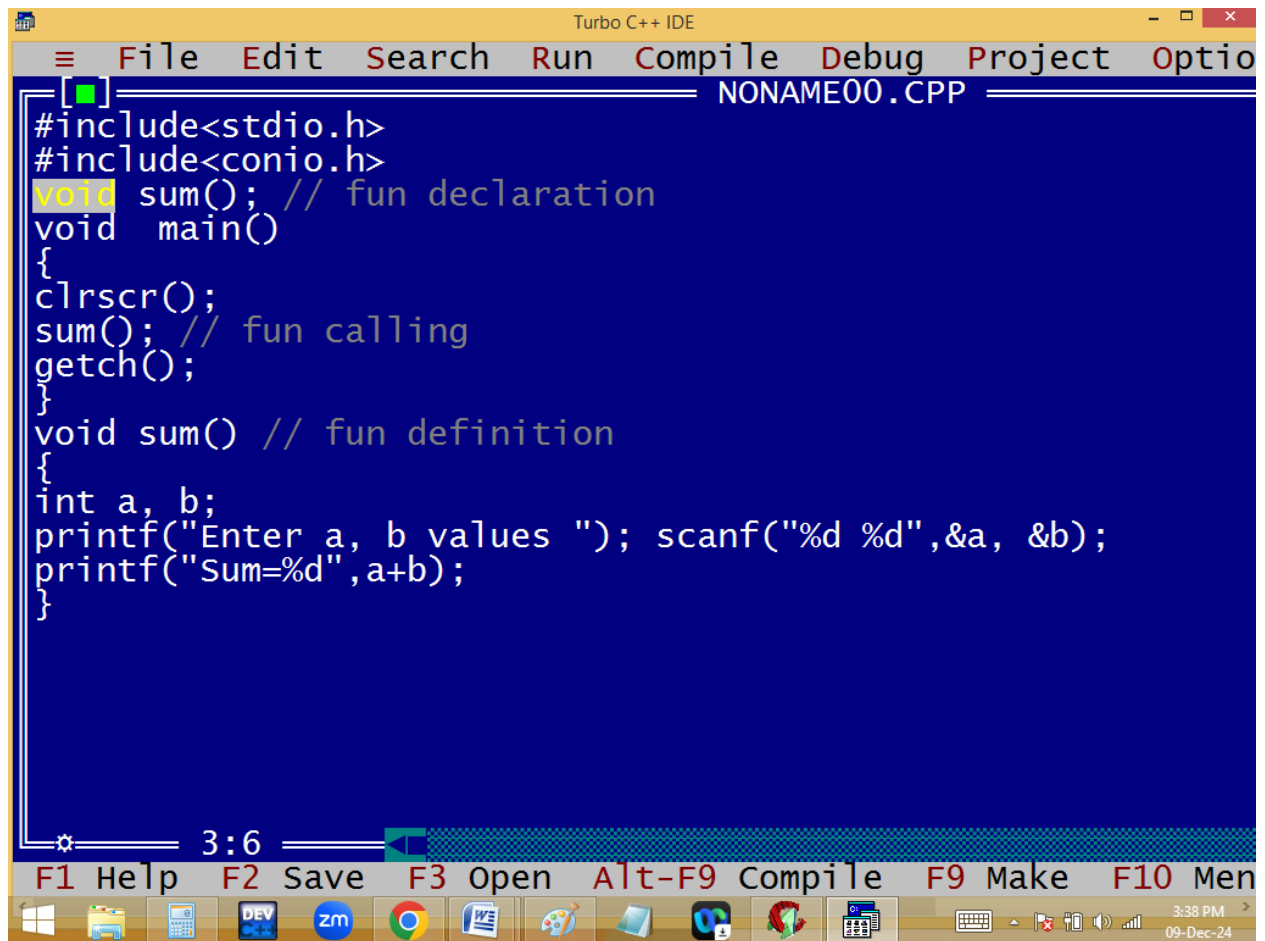


The image shows a screenshot of the Turbo C++ IDE. The title bar reads "Turbo C++ IDE". The menu bar includes "File", "Edit", "Search", "Run", "Compile", "Debug", "Project", and "Options". Two files are open: "NONAME03.CPP" and "NONAME04.CPP". The editor displays the following C++ code:

```
[■]  
#include<stdio.h>  
#include<conio.h>  
void welcome(void); // fun declaration or prototype  
void main()  
{  
clrscr();  
welcome(); // fun calling  
getch();  
}  
void welcome(void) // fun definition  
{  
puts("welcome to user defined functions");  
}
```

The status bar at the bottom shows the time "10:38" and a cursor icon. Below the IDE window is a Windows taskbar with icons for File Explorer, Calculator, DEV C++, Zoom, Google Chrome, Word, Paint, and others. The taskbar also displays the system clock as "3:35 PM" and the date "09-Dec-24".

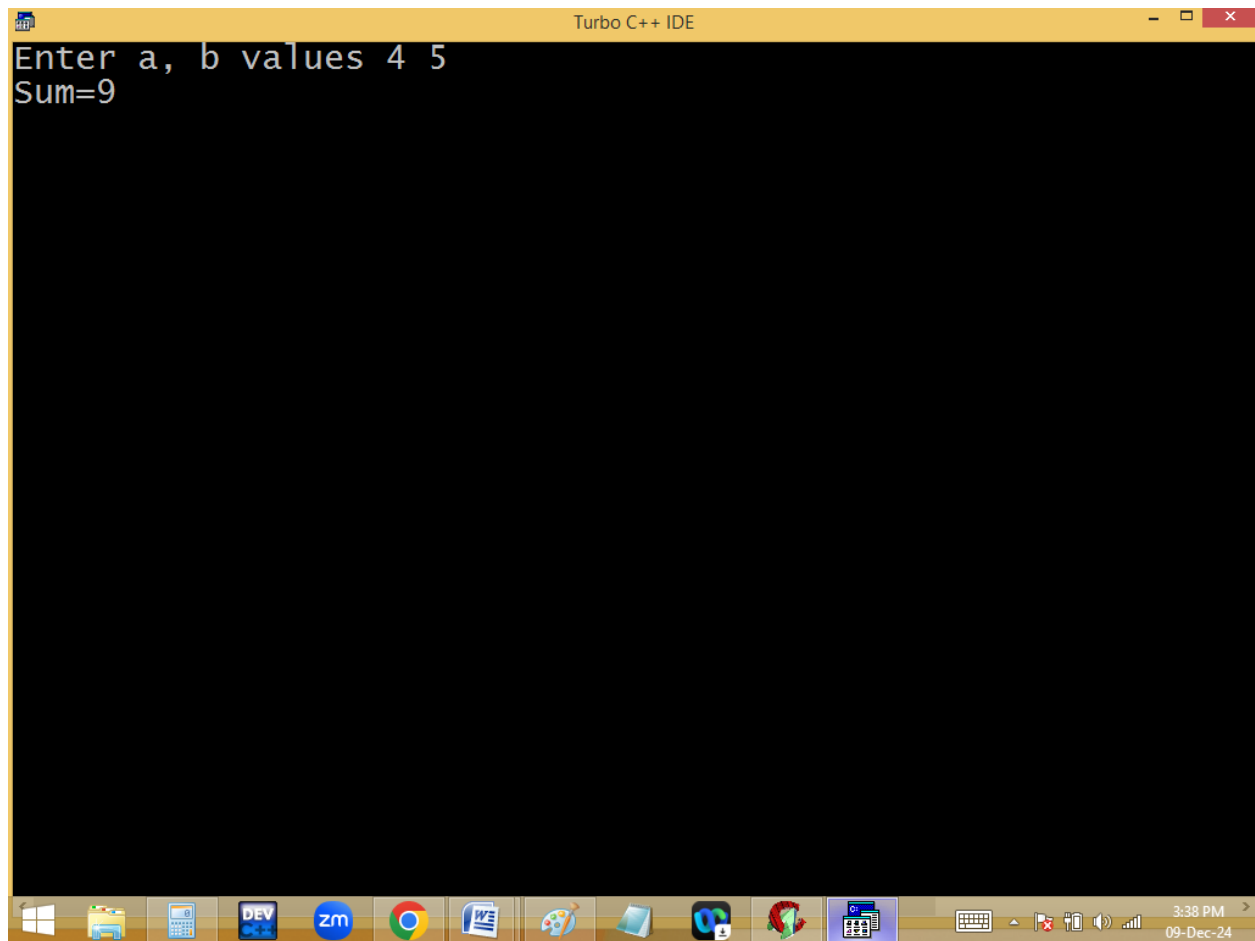




The image shows a screenshot of the Turbo C++ IDE. The title bar at the top reads "Turbo C++ IDE". The menu bar includes "File", "Edit", "Search", "Run", "Compile", "Debug", "Project", and "Options". The file name "NONAME00.CPP" is displayed in the top right. The code editor has a dark blue background with white text. The code is as follows:

```
#include<stdio.h>
#include<conio.h>
void sum(); // fun declaration
void main()
{
    clrscr();
    sum(); // fun calling
    getch();
}
void sum() // fun definition
{
    int a, b;
    printf("Enter a, b values "); scanf("%d %d",&a, &b);
    printf("Sum=%d",a+b);
}
```

The status bar at the bottom shows the cursor position "3:6" and a series of function key shortcuts: "F1 Help", "F2 Save", "F3 Open", "Alt-F9 Compile", "F9 Make", and "F10 Men". The Windows taskbar is visible at the very bottom, showing icons for the Start menu, File Explorer, Calculator, DEV C++, ZOOM, Google Chrome, Notepad, Paint, and other applications. The system clock in the bottom right corner indicates the time is 3:38 PM on 09-Dec-24.

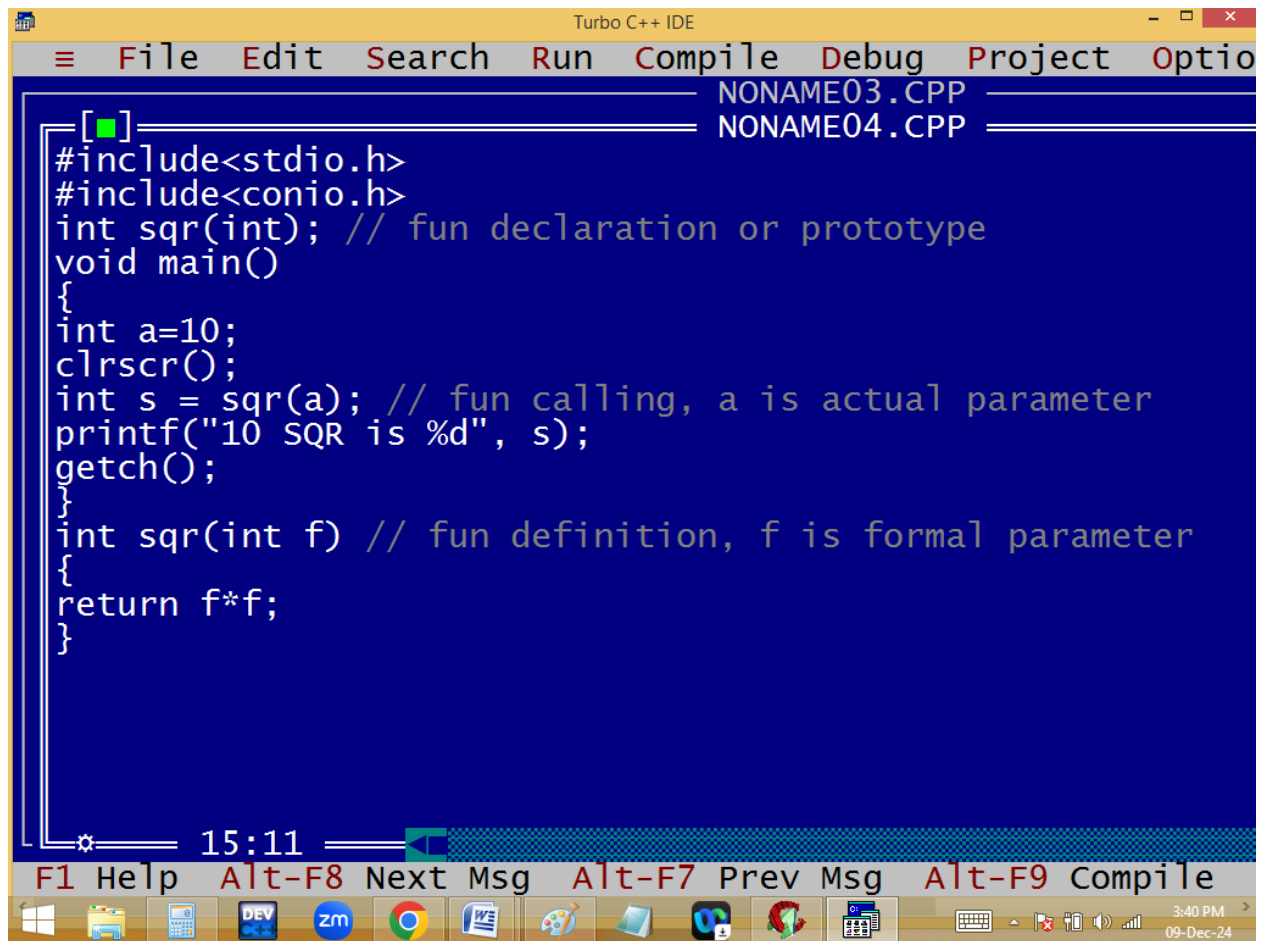


The screenshot shows the Turbo C++ IDE window. The title bar reads "Turbo C++ IDE". The main text area contains the following code:

```
Enter a, b values 4 5  
Sum=9
```

The Windows taskbar is visible at the bottom, showing various application icons and the system clock indicating 3:38 PM on 09-Dec-24.

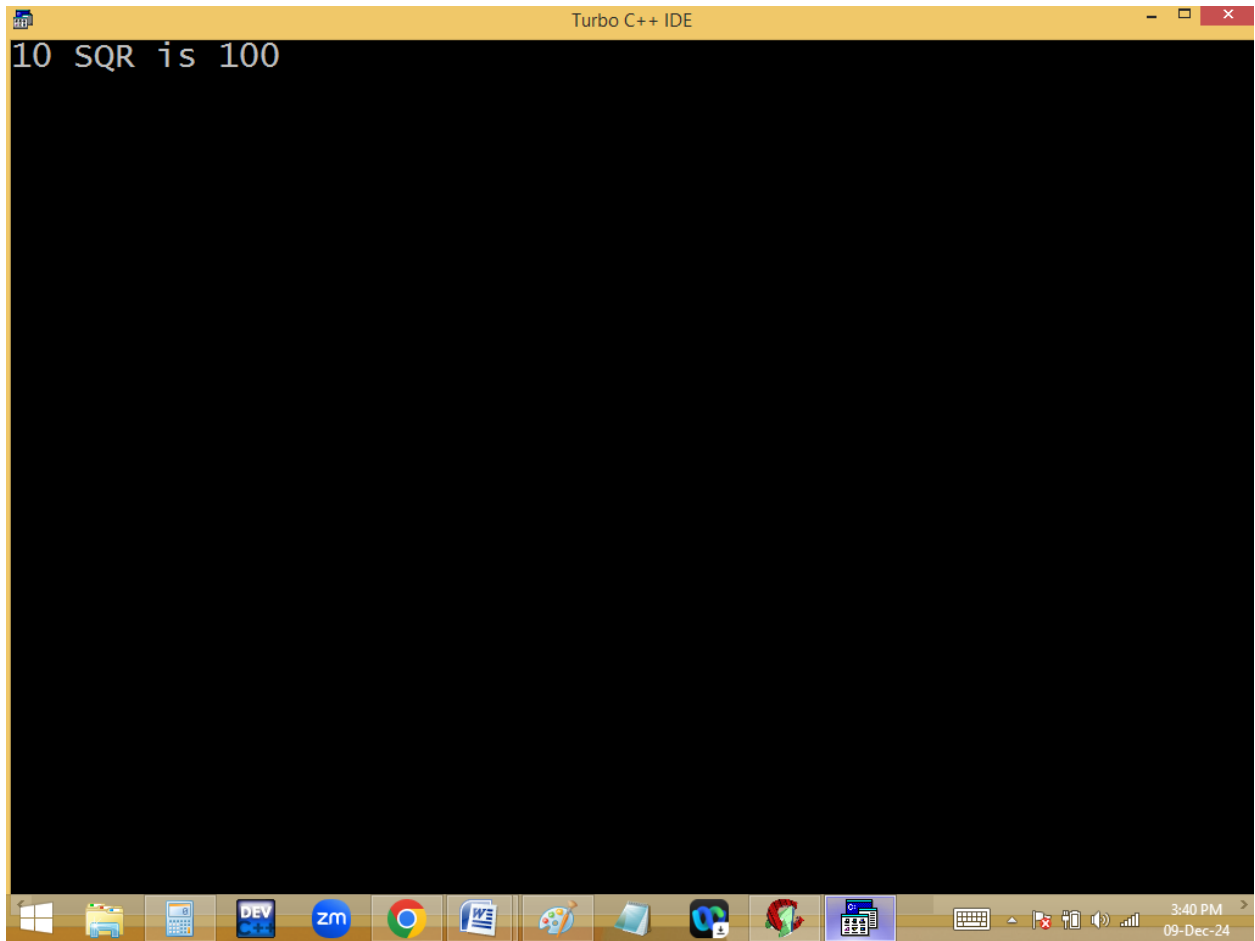
With arguments, with return value:

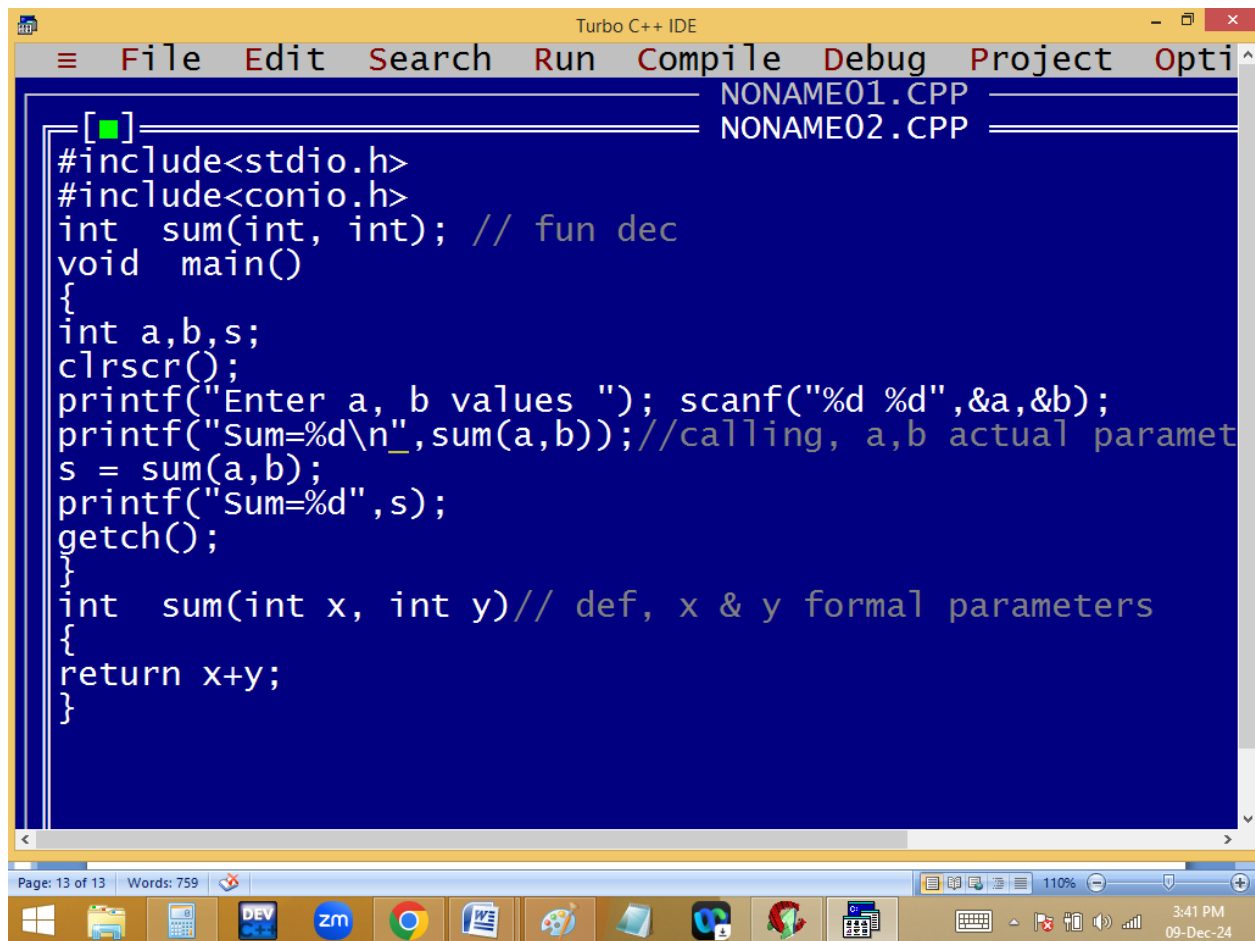


The image shows a screenshot of the Turbo C++ IDE. The title bar at the top reads "Turbo C++ IDE". The menu bar includes "File", "Edit", "Search", "Run", "Compile", "Debug", "Project", and "Options". The file list on the right shows "NONAME03.CPP" and "NONAME04.CPP". The main editor area has a blue background and contains the following C++ code:

```
[■]
#include<stdio.h>
#include<conio.h>
int sqr(int); // fun declaration or prototype
void main()
{
    int a=10;
    clrscr();
    int s = sqr(a); // fun calling, a is actual parameter
    printf("10 SQR is %d", s);
    getch();
}
int sqr(int f) // fun definition, f is formal parameter
{
    return f*f;
}
```

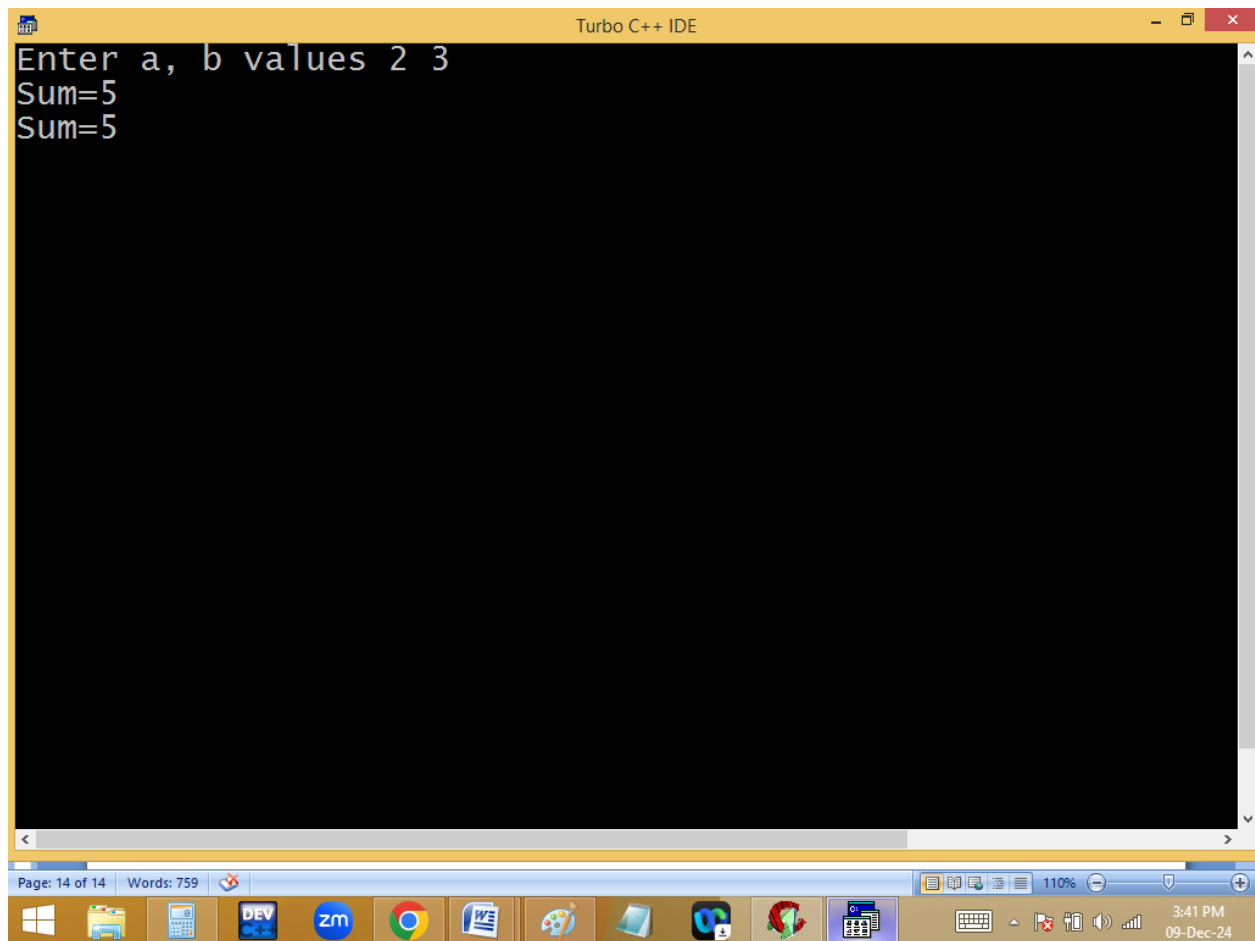
At the bottom of the editor, a status bar shows the time "15:11" and a cursor icon. Below the editor is a command bar with the following shortcuts: "F1 Help", "Alt-F8 Next Msg", "Alt-F7 Prev Msg", and "Alt-F9 Compile". The Windows taskbar at the very bottom shows various application icons, including the Start button, File Explorer, Calculator, DEV C++, ZOOM, Google Chrome, Word, Paint, and others. The system clock in the bottom right corner indicates "3:40 PM" and "09-Dec-24".

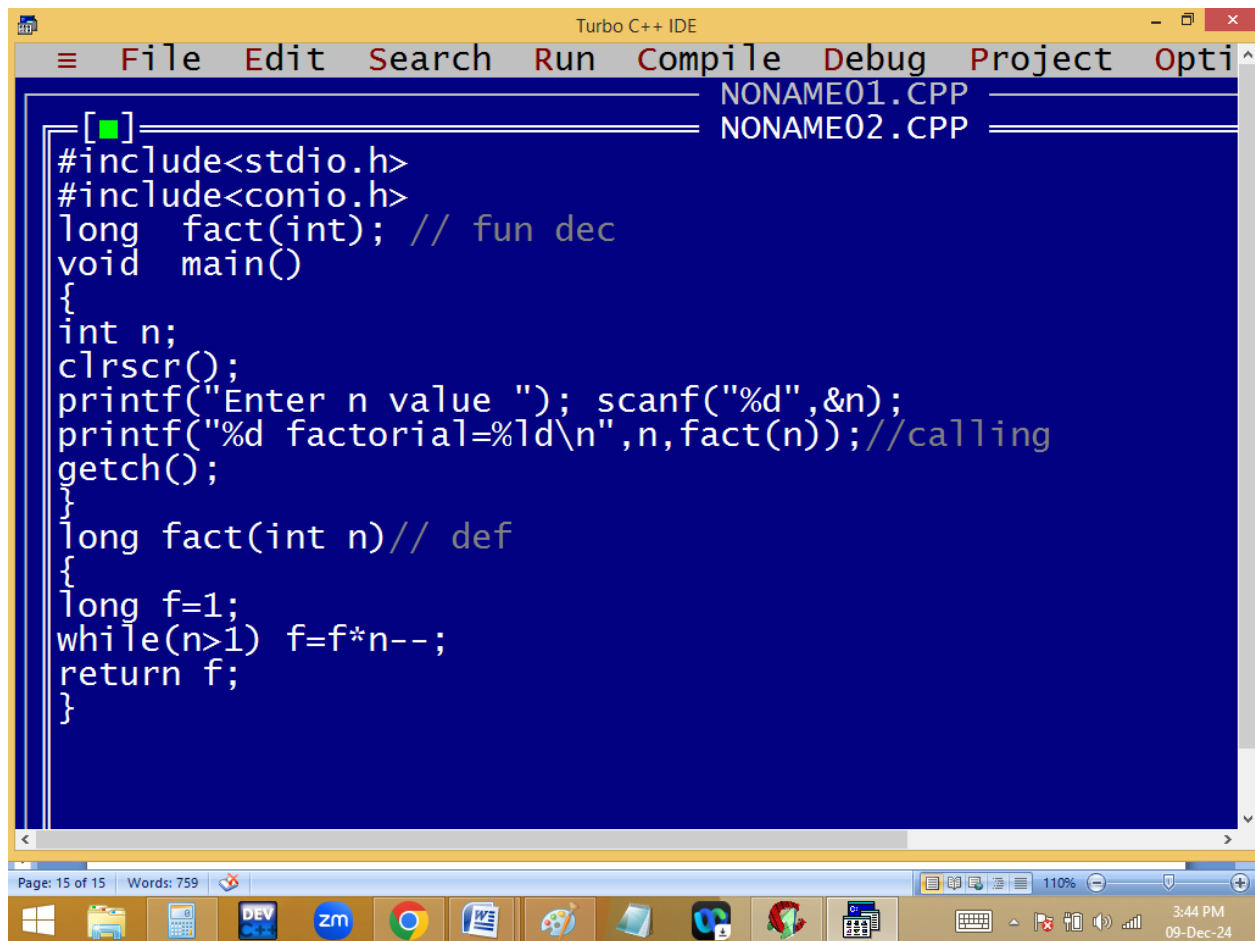




The image shows a screenshot of the Turbo C++ IDE. The title bar at the top reads "Turbo C++ IDE". The menu bar includes "File", "Edit", "Search", "Run", "Compile", "Debug", "Project", and "Options". The main window displays a C++ program in a dark blue editor. The code defines a function `sum` and a `main` function. The `main` function prompts the user to enter two values, `a` and `b`, and then prints their sum. The `sum` function is defined below `main` and takes two integers as arguments, returning their sum. The status bar at the bottom indicates "Page: 13 of 13" and "Words: 759". The Windows taskbar at the very bottom shows various icons, including the Start button, File Explorer, Calculator, and several application icons, along with the system clock showing 3:41 PM on 09-Dec-24.

```
[■]
#include<stdio.h>
#include<conio.h>
int sum(int, int); // fun dec
void main()
{
    int a,b,s;
    clrscr();
    printf("Enter a, b values "); scanf("%d %d",&a,&b);
    printf("Sum=%d\n",sum(a,b)); //calling, a,b actual paramet
    s = sum(a,b);
    printf("Sum=%d",s);
    getch();
}
int sum(int x, int y) // def, x & y formal parameters
{
    return x+y;
}
```





The image shows a screenshot of the Turbo C++ IDE. The title bar reads "Turbo C++ IDE". The menu bar includes "File", "Edit", "Search", "Run", "Compile", "Debug", "Project", and "Options". The file list on the right shows "NONAME01.CPP" and "NONAME02.CPP". The main editor window contains the following C++ code:

```
[■]
#include<stdio.h>
#include<conio.h>
long fact(int); // fun dec
void main()
{
    int n;
    clrscr();
    printf("Enter n value "); scanf("%d",&n);
    printf("%d factorial=%ld\n",n,fact(n)); //calling
    getch();
}
long fact(int n) // def
{
    long f=1;
    while(n>1) f=f*n--;
    return f;
}
```

The status bar at the bottom indicates "Page: 15 of 15" and "Words: 759". The Windows taskbar at the bottom shows various icons including Windows, File Explorer, Calculator, DEV C++, Zoom, Chrome, Word, Paint, and others. The system clock shows "3:44 PM" and "09-Dec-24".

Function with arguments, without return value:

Turbo C++ IDE

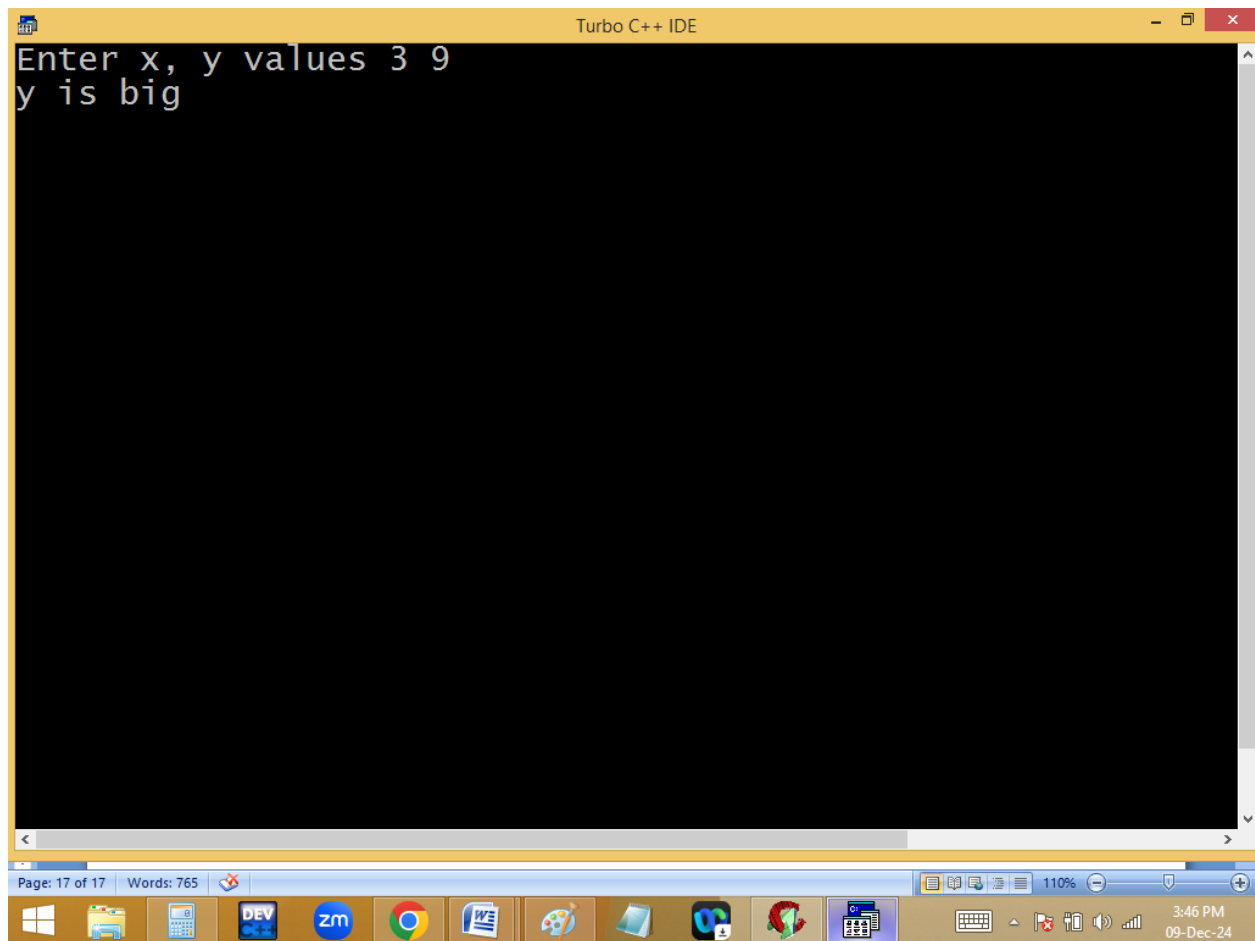
File Edit Search Run Compile Debug Project Opti

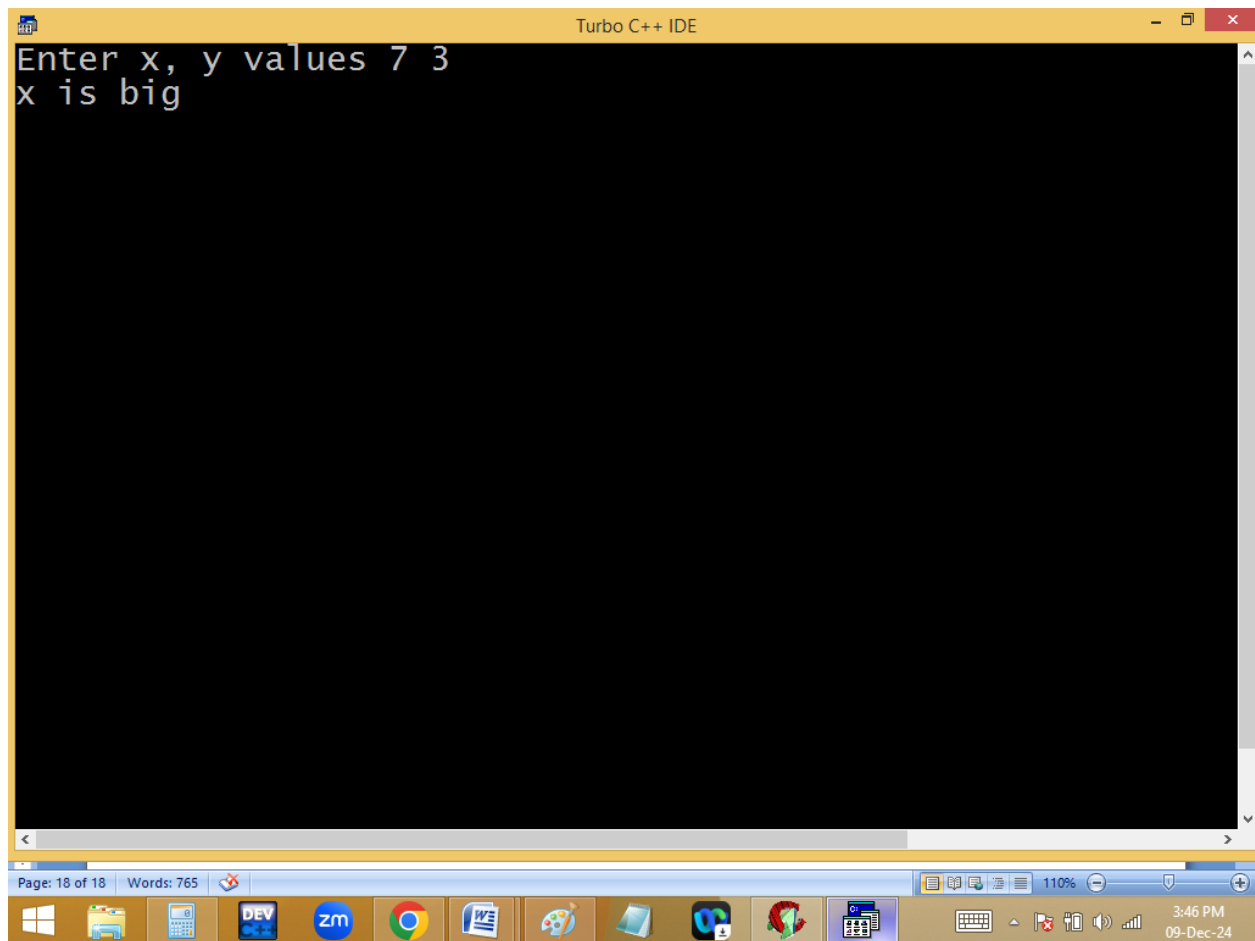
NONAME01.CPP
NONAME02.CPP
NONAME05.CPP

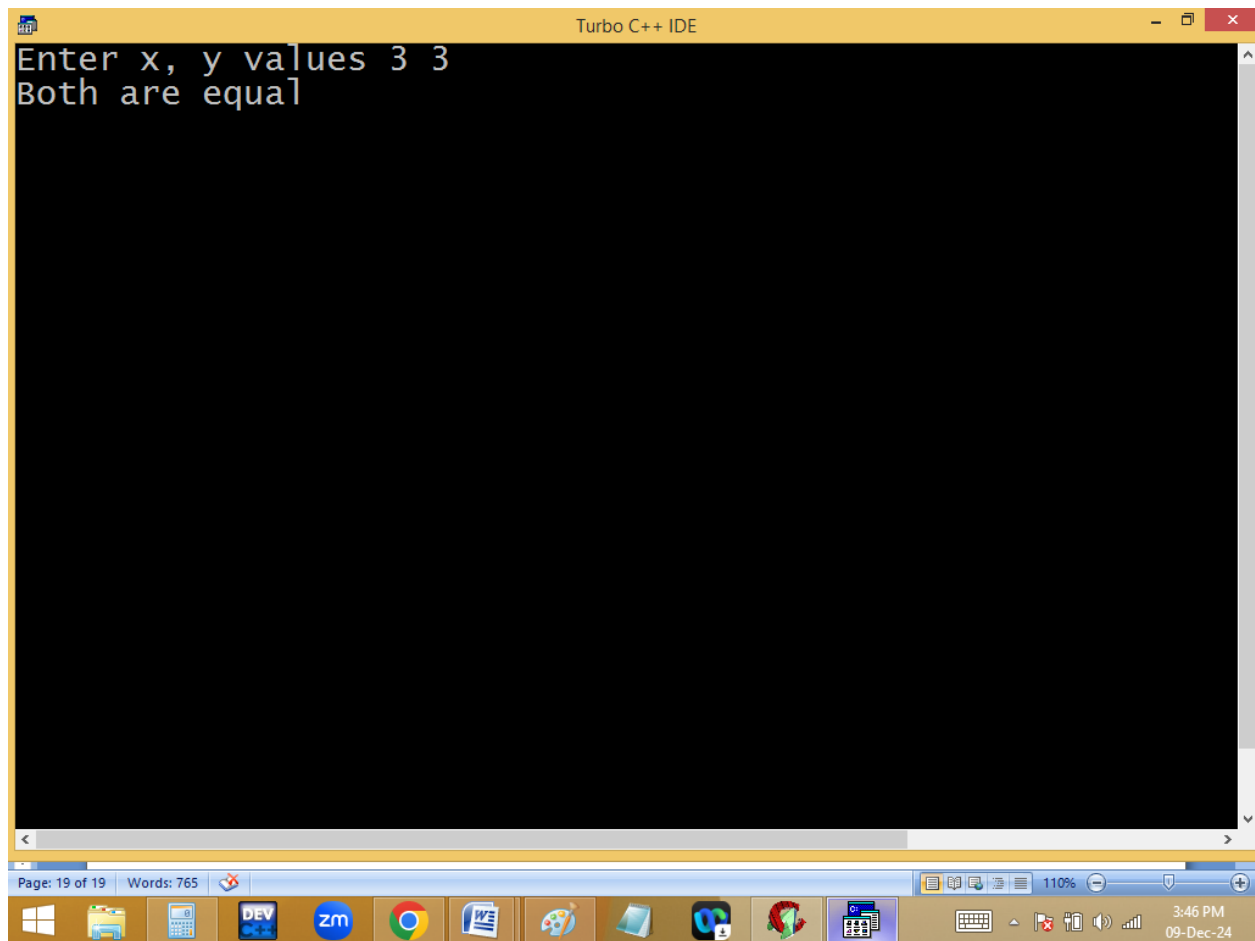
```
[ ]
#include<stdio.h>
#include<conio.h>
void max(int, int);
void max(int x, int y)
{
puts(x>y?"x is big" : y>x?"y is big":"Both are equal");
}
void main()
{
int x,y;
clrscr();
printf("Enter x, y values "); scanf("%d %d",&x, &y);
max(x,y);
getch();
}_
```

Page: 16 of 16 Words: 765 110%

3:46 PM
09-Dec-24







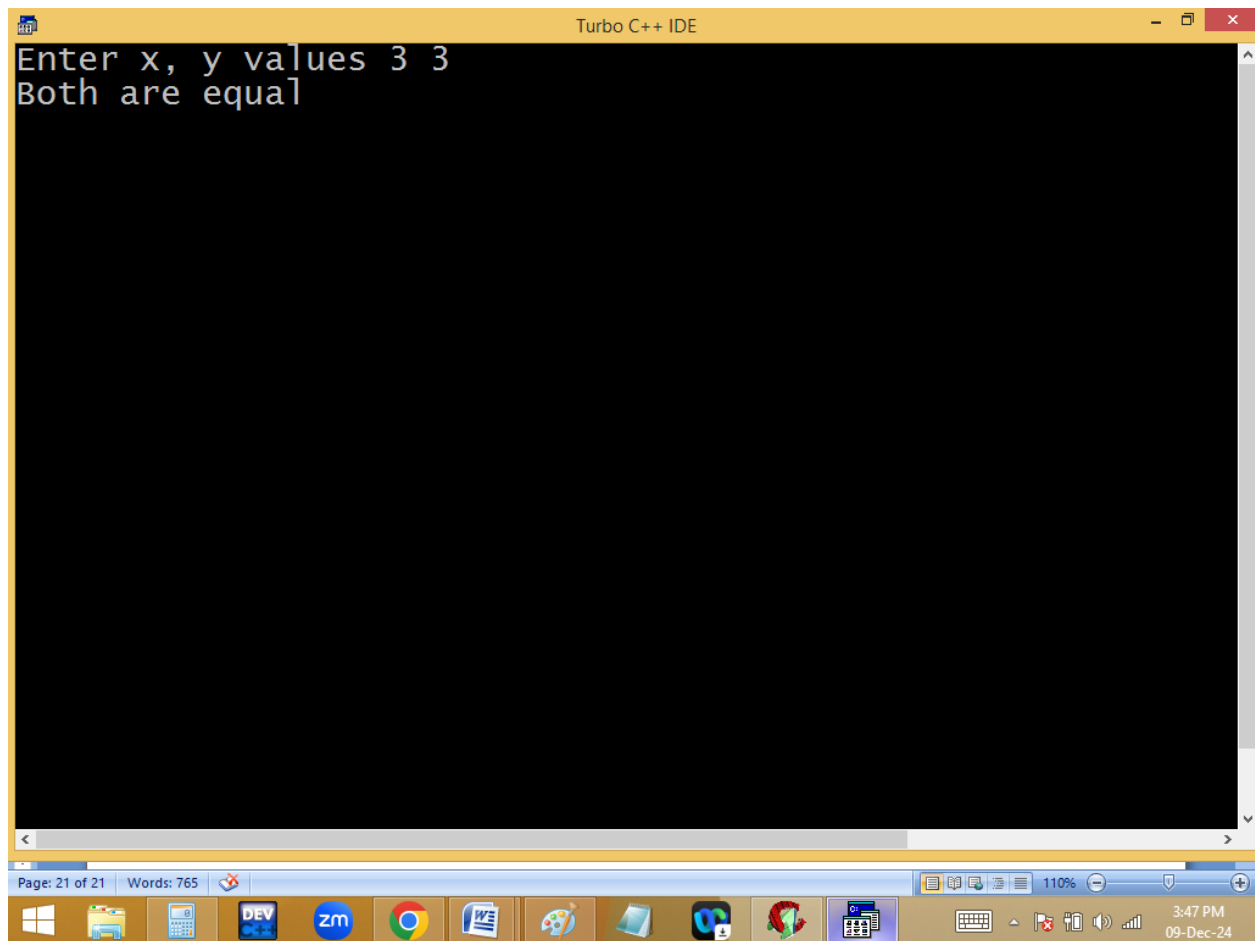
Turbo C++ IDE

File Edit Search Run Compile Debug Project Opti

NONAME03.CPP

```
#include<stdio.h>
#include<conio.h>
void sum(int, int); // fun dec
void sum(int a, int b)// fun definition, a&b formal par
{
printf("Sum=%d\n",a+b);
}
void main()
{
int a,b;
clrscr();
printf("Enter a, b values "); scanf("%d%d",&a,&b);
sum(a,b);// fun calling, a&b actual para
getch();
}
```

Page: 20 of 20 Words: 765 110% 3:47 PM 09-Dec-24



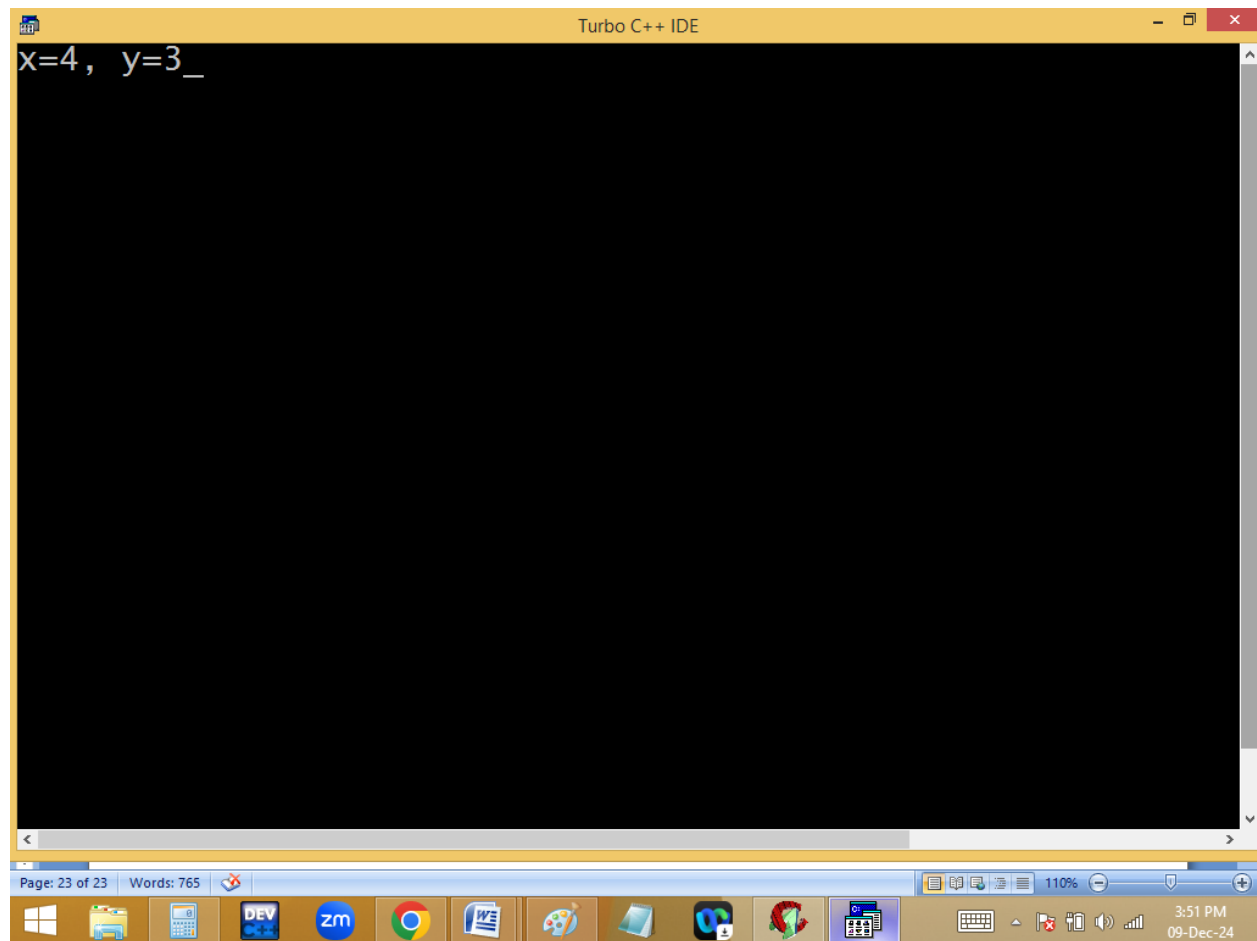
Turbo C++ IDE

File Edit Search Run Compile Debug Project Opti

NONAME06.CPP

```
#include<stdio.h>
#include<conio.h>
void show(int x, int y) // def
{
printf("x=%d, y=%d",x,y);
}
void main()
{
void show(int, int); // dec
int a=3;
clrscr();
show(a++, a++);
getch();
}
// In fun calling parameters passed from right to left
```

Page: 22 of 22 Words: 765 110% 3:51 PM 09-Dec-24



Function without arguments, with return value:

Turbo C++ IDE

File Edit Search Run Compile Debug Project Opti

NONAME01.CPP

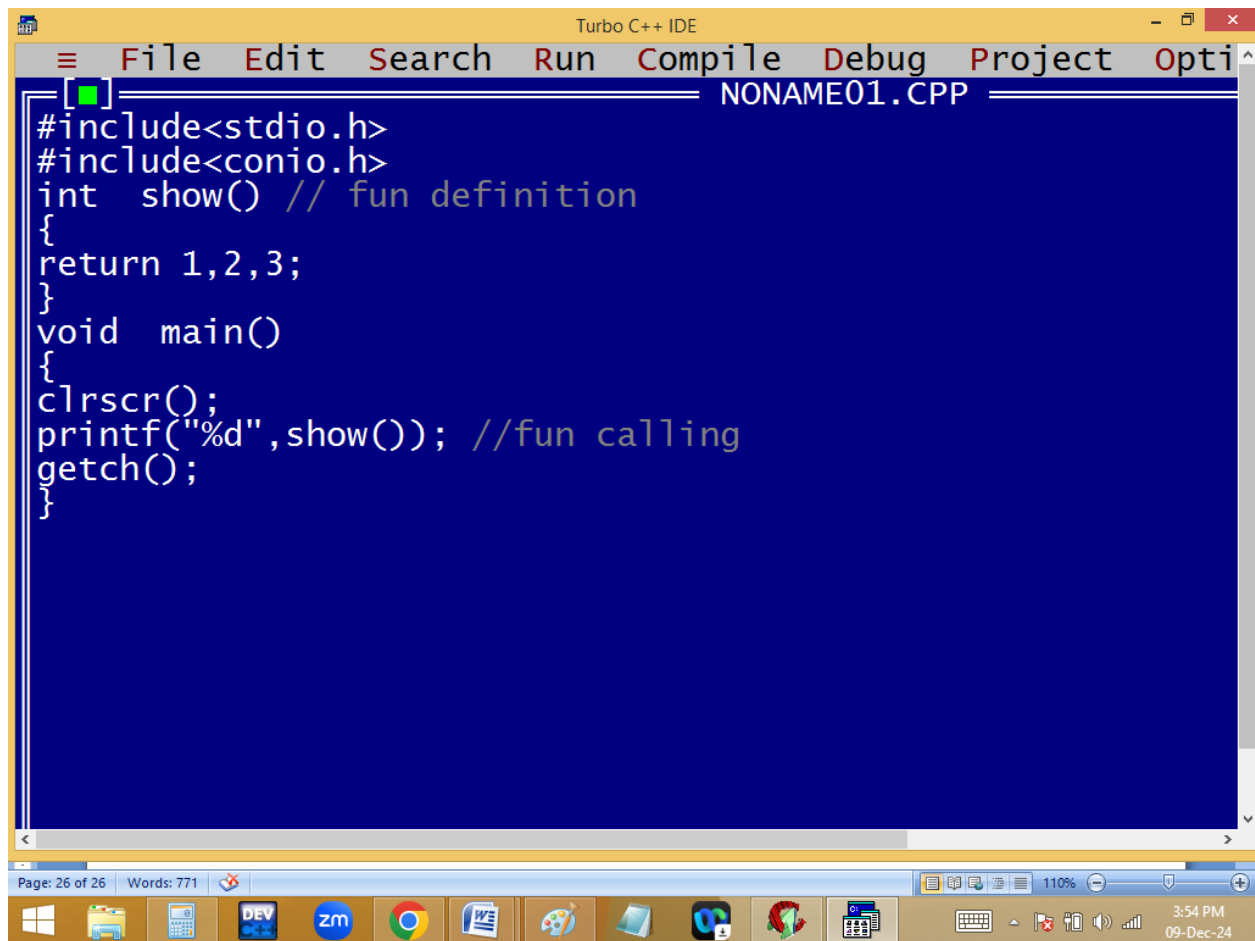
```
#include<stdio.h>
#include<conio.h>
int sum(void); // fun declaration
void main() // caller
{
clrscr();
int s = sum(); // callie , fun calling
printf("Sum=%d\n",s);
printf("Sum=%d\n",sum()); // calling
getch();
}
int sum() // fun definition
{
int a,b;
printf("Enter a, b values "); scanf("%d %d",&a, &b);
return a+b;
}
```

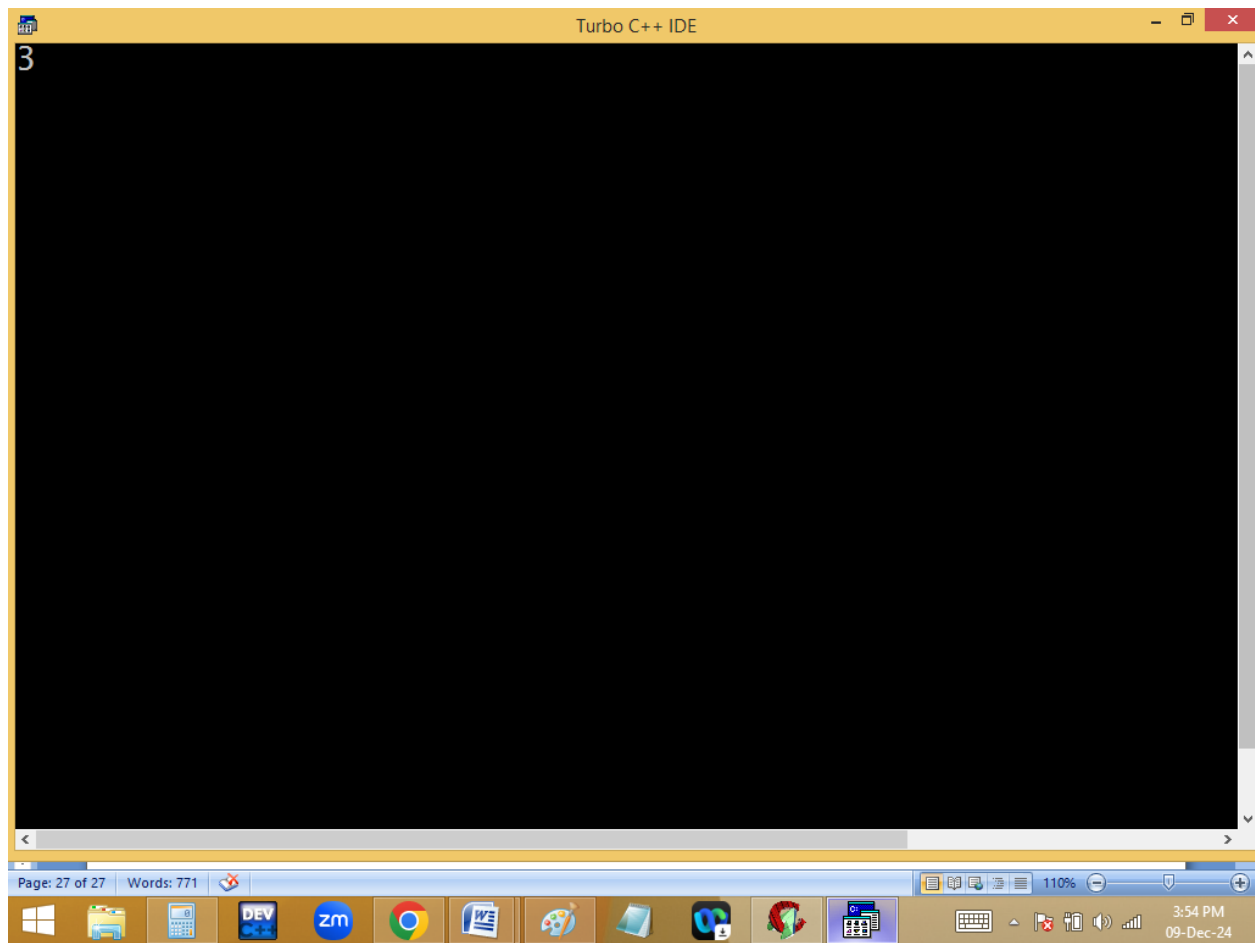
Page: 24 of 24 Words: 771 110% 09-Dec-24 3:52 PM

```
Turbo C++ IDE
Enter a, b values 1 2
Sum=3
Enter a, b values 3 4
Sum=7
```

Page: 25 of 25 | Words: 771 | 110%

3:52 PM
09-Dec-24





Turbo C++ IDE

File Edit Search Run Compile Debug Project Opti

NONAME01.CPP

```
#include<stdio.h>
#include<conio.h>
int  show() // fun definition
{
return 1,2,3;
return 2,3,4;
return 3,4,5;_
}
void  main()
{
clrscr();
printf("%d",show()); //fun calling
getch();
}
```

Page: 28 of 28 Words: 771 110% 3:55 PM 09-Dec-24

